

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
РАДІОЕЛЕКТРОНІКИ

Кафедра програмної інженерії

ЗВІТ

з лабораторної роботи № 4

«Рефакторинг коду та проведення Code Review

за індустріальними стандартами»

з дисципліни

«Основи програмної інженерії»

Виконала:

Ст. групи ПЗП-25-2 №16

Шестак Ілона Олександрівна

Викладач:

Гребенюк В'ячеслав Олександрович

Харків 2026

4.1 МЕТА РОБОТИ

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

ХІД ВИКОНАННЯ РОБОТИ

1. Результати Code Review одnogрупника.

№	Рядок коду	Проблема	Категорія	Рекомендація
1	Рядок 1: import datetime	Модуль datetime імпортується повністю, хоча в кодi використовується лише виклик datetime.datetime.now(), що створює тавтологію.	Dead Code (Зайвий імпорт)	Замінити на from datetime import datetime, а в кодi викликати просто datetime.now().
2	Рядок 21: if len(password) <= 6:	Використано нестрогу нерівність <= 6. Через це валідний пароль довжиною рівно в 6 символів буде помилково заблоковано. Також число 6 є «магічним».	Логічна помилка (Boundary Bug) + Magic Numbers	Змінити умову на len(password) < 6 або винести 6 в константу MIN_PASSWORD_LENGTH.
3	Рядок 39: if len(new_nickname) < 3 or ...	Числа 3 та 15 вбудовані прямо в логіку валідації нікнейму як жорсткі літерали без пояснення контексту.	Magic Numbers	Винести ліміти у константи класу або модуля: MIN_NICK_LEN = 3 та MAX_NICK_LEN = 15.

4	Рядок 68: <code>if len(content) == 0:</code>	Перевірка не враховує пробільні символи. Публікація з одних пробілів (" ") успішно створиться, засмічуючи базу даних порожнім контентом.	Порушення обробки граничних випадків	Застосувати очищення рядка від пробілів з обох боків: <code>if len(content.strip()) == 0:</code> .
5	Рядок 71: <code>if len(content) > 280:</code>	Ліміт довжини посту у 280 символів прописаний жорстко. При зміні бізнес-логіки доведеться шукати це число по всьому проєкту.	Magic Numbers	Винести значення в іменовану константу класу Post: <code>MAX_POST_LENGTH = 280</code> .

2. Посилання на PR з коментарями Code Review:

<https://github.com/qwemaqqe123/FaceCloud/pull/1#pullrequestreview-4412447388>

3. Результати рефакторингу власного коду.

БУЛО:

```
int maxPostLengthLimit = 500;

if (user == null) throw new ArgumentNullException(nameof(user));
```

СТАЛО:

```
if (user == null) throw new ArgumentNullException(nameof(user));
```

ЧОМУ: Змінна `maxPostLengthLimit` була оголошена, але ніде не використовувалася в методі, що викликало попередження компілятора (warning CS0219) та було зафіксовано SonarLint. Також закоментовані дебаг-принти засмічують кодову базу (Dead Code). Уся історія розробки зберігається в Git, тому мертвий код і зайві змінні було повністю видалено для покращення супроводжуваності проєкту.

БУЛО:

// У методі AddWarning:

```
if (user.Warnings >= 3) { ... }
```

// У методі CanPostContent:

```
if (content.Length > 500) { ... }
```

СТАЛО:

```
public class ModerationService
```

```
{
```

```
    private const int MaxWarningsBeforeBan = 3;
```

```
    private const int MaxPostLength = 500;
```

```
    // Використання констант у методах:
```

```
    if (user.Warnings >= MaxWarningsBeforeBan) { ... }
```

```
    if (content.Length > MaxPostLength) { ... }
```

```
}
```

ЧОМУ: Пряме використання числових літералів 3 та 500 (Magic Numbers) є грубим порушенням індустріальних стандартів якості коду. При зміні правил модерації платформи (наприклад, збільшення ліміту символів) довелося б шукати ці числа вручну по всьому коду. Винесення їх у константи підвищує читабельність та забезпечує єдину точку конфігурації проекту.

БУЛО:

```
public class User
```

```
{
```

```
    public string Username { get; set; }
```

```
    public int Warnings { get; set; }
```

```
    public bool IsBanned { get; set; }
```

```
}
```

СТАЛО:

```
public class User
{
    public string Username { get; set; }

    public int Warnings { get; private set; } // Закрытый сеттер

    public bool IsBanned { get; private set; } // Закрытый сеттер

    public void IncrementWarnings() => Warnings++;

    public void SetBanned(bool banned) => IsBanned = banned;

    public void ResetWarnings() => Warnings = 0;
}
```

ЧОМУ: Будь-який зовнішній код міг випадково або навмисно встановити негативну кількість варнів чи зняти бан в обхід сервісу модерації. Заміна сеттерів на `private set` та керування станом через методи класу захищає об'єкт від некоректних модифікацій.

4. Звіт регресійного тестування.

```
PS D:\Coding\GitHub\NURE\opi_1-2\bse-lr1-Shestak\FaceCloudTests> dotnet test /p:CollectCoverage=true
Restore complete (0,4s)
    FaceCloudApp net10.0 succeeded (0,2s) → D:\Coding\GitHub\NURE\opi_1-2\bse-lr1-Shestak\FaceCloudApp\bin\Debug\net10.0\FaceCloudApp.dll
    FaceCloudTests net10.0 succeeded (0,1s) → bin\Debug\net10.0\FaceCloudTests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.8)
[xUnit.net 00:00:00.08]   Discovering: FaceCloudTests
[xUnit.net 00:00:00.12]     Discovered: FaceCloudTests
[xUnit.net 00:00:00.14]       Starting: FaceCloudTests
[WARN] ##### : ##### 
[Moderation] ##### User: X 皆 ##### 3 #9.
[xUnit.net 00:00:00.21]   Finished: FaceCloudTests
    FaceCloudTests net10.0                                     GenerateCoverageResult (1,4s)

+-----+-----+-----+-----+
| Module | Line | Branch | Method |
+-----+-----+-----+-----+
| FaceCloudApp | 92.3% | 95.45% | 90% |
+-----+-----+-----+-----+

+-----+-----+-----+-----+
|      | Line | Branch | Method |
+-----+-----+-----+-----+
| Total | 92.3% | 95.45% | 90% |
+-----+-----+-----+-----+
| Average | 92.3% | 95.45% | 90% |
+-----+-----+-----+-----+

FaceCloudTests test net10.0 succeeded (1,4s)

Test summary: total: 11; failed: 0; succeeded: 11; skipped: 0; duration: 0,8s
Build succeeded in 2,7s
PS D:\Coding\GitHub\NURE\opi_1-2\bse-lr1-Shestak\FaceCloudTests>
```

5. Порівняння метрик «ДО / ПІСЛЯ».

Метрика покриття	Значення ДО	Значення ПІСЛЯ
Line Coverage	91.3%	92.3%
Branch Coverage	95.45%	95.45%
Method Coverage	83.3%	90.0%

6. Підсумкова рефлексія.

Досягнення високого показника у 92.3% покриття рядків є чудовим підтвердженням того, що проведений рефакторинг пройшов успішно й не залишив «сліпих зон» у бізнес-логіці програми. Очищення коду від закоментованих рядків та невикористаних змінних дозволило зробити програму лаконічнішою, що автоматично покращило відносну щільність тестування. Така висока метрика гарантує стабільність роботи системи модерації та значно спрощує її подальший супровід. Цей результат наочно доводить, що систематичне поліпшення архітектури безпосередньо підвищує надійність та якість кінцевого програмного продукту.

Висновки: У результаті виконання лабораторної роботи набуто практичних навичок проведення перевірки коду (Code Review) з використанням інструментів спільної роботи GitHub Pull Requests. Проведено детальний аналіз вихідного коду для виявлення дефектів проектування (code smells) та виконано успішний рефакторинг програмного модуля із забезпеченням суворої інкапсуляції та усуненням застарілих конструкцій. Проведене регресійне модульне тестування підтвердило повну цілісність бізнес-логіки системи та продемонструвало покращення метрик покриття коду.